

自学手册制作指南

第12章 总览

本章核心目标

- 理解类中如何使用 `new` 和 `delete` 实现动态内存分配。
- 掌握复制构造函数、赋值运算符的重载方法。
- 理解浅拷贝与深拷贝的区别及其重要性。
- 理解静态类成员的使用方式。
- 使用指针操作对象。
- 完善类的内存管理，避免常见错误如“重复释放内存”。

12.1 动态内存和类

12.1.1 动态内存基础

定义

C++提供 `new` 和 `delete` 运算符用于在程序运行时动态地分配和释放内存。这种机制与人们“吃饭时再决定吃什么”的行为类似，适合处理运行时才能确定的数据量。

示例代码

cpp

```
int* p = newint;    // 分配一个int
*p = 5;
delete p;           // 释放内存
```

12.1.2 StringBad 类：错误示例学习

定义

`StringBad` 类是一个用于教学的示例类，通过它可以学习动态内存分配的常见错误。

cpp

```
classStringBad {
private:
    char* str;
    int len;
    staticint num_strings;
public:
    StringBad(constchar* s);
    StringBad();
    ~StringBad();
    friend std::ostream& operator<<(std::ostream& os, const StringBad& st);
};
```

关键问题

1. 使用 **new** 分配内存，但没有定义复制构造函数或赋值运算符。
2. 类中使用 **char*** 指针，但按值传递对象或赋值时发生“浅拷贝”。
3. 字符串数据没有独立副本，多个对象共享指针，导致多次释放同一内存。

类型	定义	示例
浅拷贝	仅复制指针地址	<code>a.str = b.str;</code>
深拷贝	分配新内存并复制字符串内容	<code>str = new char[strlen(b.str) + 1]; strcpy(str, b.str);</code>

示例：深拷贝复制构造函数

cpp

```
StringBad::StringBad(const StringBad& s) {  
    len = s.len;  
    str = newchar[len + 1];  
    strcpy(str, s.str);  
    num_strings++;  
}
```

12.1.4 赋值运算符的正确实现

错误方式

cpp

```
knot = headline1; // 默认浅拷贝，导致双重释放
```

正确实现

cpp

```
StringBad& StringBad::operator=(const StringBad& st) {  
    if (this == &st)  
        return *this;  
    delete[] str;  
    len = st.len;  
    str = newchar[len + 1];  
    strcpy(str, st.str);  
    return *this;  
}
```

12.2 改进后的 String 类

12.2.1 功能增强

- 添加复制构造函数
- 添加赋值运算符
- 提供字符串长度、比较、索引访问等功能
- 引入静态成员函数 `HowMany()` 跟踪对象数量

cpp

```
classString {
private:
    char* str;
    int len;
    staticint num_strings;
public:
    String(constchar* s);
    String();
    String(const String& s);
    ~String();
    String& operator=(const String& s);
    String& operator=(constchar* s);
    intlength()const;
    char& operator[](int i);
    staticintHowMany();
};
```

cpp

```
int String::num_strings = 0;
```

cpp

```
int count = String::HowMany();
```

cpp

```
friend std::ostream& operator<<(std::ostream& os, const String& st);
friend std::istream& operator>>(std::istream& is, String& st);
```

错误类型	原因	解决方案
浅拷贝共享字符串	默认复制构造函数未分配新内存	显式定义复制构造函数，执行深拷贝
双重释放或野指针	多个对象指向同一内存	每个对象持有自己的内存副本
忘记释放动态内存	析构函数中未使用 <code>delete</code>	在析构函数中释放 <code>new</code> 分配的内存
自赋值导致崩溃	对象赋值给自身时释放了自身内存	在赋值运算符中检查 <code>this == &st</code>

下一章预告：对象数组与动态内存更深入

是否需要我将整章内容都按此格式完整展开？还是只需要部分章节？也可以将内容输出为 Word/PDF 或便于打印的格式。请告诉我你的偏好。

到目前为止，此对话有帮助吗？

C++自学手册：第12章 类与动态内存分配

ChatGPT 也可能会犯错。请核查重要信息。



好的，以下是依据你提供的《第12章 类和动态内存分配》文档内容，整理出的详细易懂的自学手册示例部分。本节内容将以结构化的形式，围绕每个核心知识点展开，适用于零基础到中级水平的C++学习者。

C++自学手册：第12章 类与动态内存分配

第12章 总览

本章核心目标

- 理解类中如何使用 `new` 和 `delete` 实现动态内存分配。
- 掌握复制构造函数、赋值运算符的重载方法。
- 理解浅拷贝与深拷贝的区别及其重要性。
- 理解静态类成员的使用方式。
- 使用指针操作对象。
- 完善类的内存管理，避免常见错误如“重复释放内存”。

12.1 动态内存和类

12.1.1 动态内存基础

定义

C++提供 `new` 和 `delete` 运算符用于在程序运行时动态地分配和释放内存。这种机制与人们“吃饭时再决定吃什么”的行为类似，适合处理运行时才能确定的数据量。

示例代码

```
int* p = new int; // 分配一个int

*p = 5;

delete p; // 释放内存
```

12.1.2 StringBad 类：错误示例学习

定义

`StringBad` 类是一个用于教学的示例类，通过它可以学习动态内存分配的常见错误。

类声明摘要

```
class StringBad {
```

```

private:

char* str;

int len;

static int num_strings;

public:

StringBad(const char* s);

StringBad();

~StringBad();

friend std::ostream& operator<<(std::ostream& os, const StringBad& st);

};

```

关键问题

1. 使用 **new** 分配内存，但没有定义复制构造函数或赋值运算符。
2. 类中使用 **char*** 指针，但按值传递对象或赋值时发生“浅拷贝”。
3. 字符串数据没有独立副本，多个对象共享指针，导致多次释放同一内存。

12.1.3 浅拷贝 vs 深拷贝

类型	定义	示例
浅拷贝	仅复制指针地址	<code>a.str = b.str;</code>
深拷贝	分配新内存并复制字符串内容	<code>str = new char[strlen(b.str) + 1]; strcpy(str, b.str);</code>

示例：深拷贝复制构造函数

```

StringBad::StringBad(const StringBad& s) {

len = s.len;

str = new char[len + 1];

strcpy(str, s.str);

num_strings++;

}

```

12.1.4 赋值运算符的正确实现

错误方式

knot = headlinel; // 默认浅拷贝，导致双重释放

正确实现

```
StringBad& StringBad::operator=(const StringBad& st) {  
  
    if (this == &st)  
  
        return *this;  
  
    delete[] str;  
  
    len = st.len;  
  
    str = new char[len + 1];  
  
    strcpy(str, st.str);  
  
    return *this;  
  
}
```

12.2 改进后的 String 类

12.2.1 功能增强

- 添加复制构造函数
- 添加赋值运算符
- 提供字符串长度、比较、索引访问等功能
- 引入静态成员函数 `HowMany()` 跟踪对象数量

类声明简化

```
class String {  
  
private:  
  
    char* str;  
  
    int len;  
  
    static int num_strings;  
  
public:
```



```
String(const char* s);

String();

String(const String& s);

~String();

String& operator=(const String& s);

String& operator=(const char* s);

int length() const;

char& operator[](int i);

static int HowMany();

};
```

12.2.2 静态成员函数使用

初始化

```
int String::num_strings = 0;
```

访问

```
int count = String::HowMany();
```

12.2.3 输入输出运算符重载

```
friend std::ostream& operator<<(std::ostream& os, const String& st);
```

```
friend std::istream& operator>>(std::istream& is, String& st);
```

12.2.4 常见错误总结

错误类型	原因	解决方案
浅拷贝共享字符串	默认复制构造函数未分配新内存	显式定义复制构造函数，执行深拷贝
双重释放或野指针	多个对象指向同一内存	每个对象持有自己的内存副本
忘记释放动态内存	析构函数中未使用 <code>delete</code>	在析构函数中释放 <code>new</code> 分配的内存

自赋值导致崩溃 对象赋值给自身时释放了自身内存 在赋值运算符中检查 `this == &st`

下一章预告：对象数组与动态内存更深入

是否需要我将整章内容都按此格式完整展开？还是只需要部分章节？也可以将内容输出为 Word/PDF 或便于打印的格式。请告诉我你的偏好。